

Gitlawb / openclaude Public

<> Code • Issues 84 🔄 Pull requests 105 💬 Discussions 🎬 Actions 📁 Projects 🛡️ Security and quality 3 📈 Insights

🔑 main ▾ 🌐 40 Branches 🏷️ 24 Tags 🔍 🔖

🔍 Go to file

Go to file

Code

⋮

🐙 3kin0x feat(query): robust multi-lingual and structural continuation nudge (#... 🗨️ ✅ 2f8aa50 · 14 hours ago 🕒

📁 .github	chore: centralize Bun version and refresh CI ...	2 weeks ago
📁 bin	Fix launcher heap setup for long sessions (#...	14 hours ago
📁 docs	feat: configure API retry backoff (#370) (#10...	14 hours ago
📁 python	fix: update vulnerable dependencies (#1149)	2 weeks ago
📁 scripts	Fix launcher heap setup for long sessions (#...	14 hours ago
📁 src	feat(query): robust multi-lingual and structu...	14 hours ago
📁 tests	Harden test isolation for smoke stability (#1...	last week
📁 vscode-extension/openclaude-vscode	Harden test isolation for smoke stability (#1...	last week
📁 web	Feat/web landing refresh (#958)	last month
📄 .bun-version	chore: centralize Bun version and refresh CI ...	2 weeks ago
📄 .dockerignore	Feat/web landing refresh (#958)	last month
📄 .env.example	feat: configure API retry backoff (#370) (#10...	14 hours ago
📄 .gitignore	fix(codex): normalize empty MCP object sch...	2 weeks ago
📄 .npmignore	Feat/web landing refresh (#958)	last month
📄 .release-please-manifest.json	chore(main): release 0.14.0 (#1217)	3 days ago
📄 ANDROID_INSTALL.md	chore: centralize Bun version and refresh CI ...	2 weeks ago
📄 CHANGELOG.md	chore(main): release 0.14.0 (#1217)	3 days ago
📄 CODE_OF_CONDUCT.md	docs: add community standard files (#257)	last month
📄 CONTRIBUTING.md	Registry-Based Integration Architecture for ...	3 weeks ago
📄 Dockerfile	chore: centralize Bun version and refresh CI ...	2 weeks ago
📄 LICENSE	hardening: isolate third-party paths and clea...	last month
📄 PLAYBOOK.md	Store provider profiles in user config (#969)	3 weeks ago
📄 README.md	docs: add Xiaomi MiMo sponsor (#1213)	last week
📄 SECURITY.md	docs: add security policy	last month
📄 bun.lock	fix: update vulnerable dependencies (#1149)	2 weeks ago
📄 package.json	chore(main): release 0.14.0 (#1217)	3 days ago
📄 release-please-config.json	ci: add secure automated release workflow (...)	last month
📄 tsconfig.json	fix(typecheck): make bun run typecheck acti...	last month

OpenClaude

OpenClaude is an open-source coding-agent CLI for cloud and local model providers.

https://github.com/Gitlawb/openclaude

1/7

Use OpenAI-compatible APIs, Gemini, GitHub Models, Codex OAuth, Codex, Ollama, Atomic Chat, and other supported backends while keepir one terminal-first workflow: prompts, tools, agents, MCP, slash commands, and streaming output.

 PR Checks

passing

release

v0.14.0

discussions

open

security

policy

license

MIT

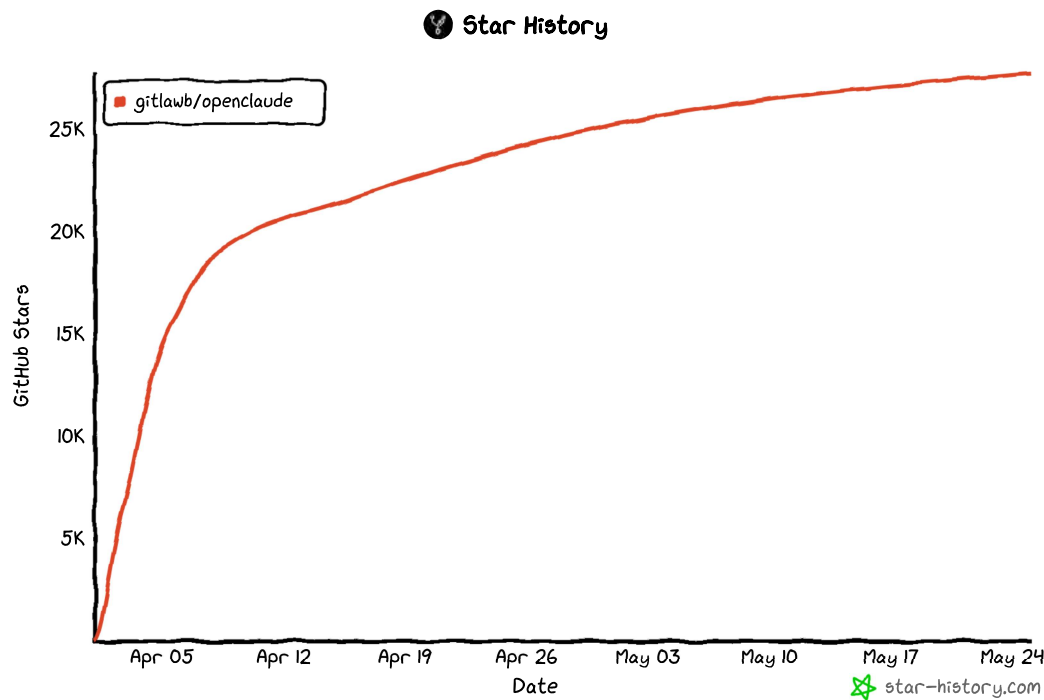
OpenClaude is also mirrored to GitLawb: gitlawb.com/node/repos/z6MkqDnb/openclaude

[Quick Start](#) | [Setup Guides](#) | [Providers](#) | [Source Build](#) | [VS Code Extension](#) | [Sponsors](#) | [Community](#)

Sponsors

			Xiaomi MIMO
GitLawb	Bankr.bot	Atomic Chat	Xiaomi MiMo

Star History



Why OpenClaude

- Use one CLI across cloud APIs and local model backends
- Save provider profiles inside the app with `/provider`
- Run with OpenAI-compatible services, Gemini, GitHub Models, Codex OAuth, Codex, Ollama, Atomic Chat, and other supported providers
- Keep coding-agent workflows in one place: bash, file tools, grep, glob, agents, tasks, MCP, and web tools
- Use the bundled VS Code extension for launch integration and theme support

Quick Start

Install

```
npm install -g @gitlawb/openclaude
```

If the install later reports `ripgrep not found`, install `ripgrep` system-wide and confirm `rg --version` works in the same terminal before startir OpenClaude.

Start

```
openclaude
```

Inside OpenClaude:

- `run /provider` for guided provider setup and saved profiles
- `run /onboard-github` for GitHub Models onboarding

Fastest OpenAI setup

macOS / Linux:

```
export CLAUDE_CODE_USE_OPENAI=1
export OPENAI_API_KEY="sk-your-key-here"
export OPENAI_MODEL=gpt-4o
```



openclaude

Windows PowerShell:

```
$env:CLAUDE_CODE_USE_OPENAI="1"
$env:OPENAI_API_KEY="sk-your-key-here"
$env:OPENAI_MODEL="gpt-4o"
```



openclaude

Fastest local Ollama setup

macOS / Linux:

```
export CLAUDE_CODE_USE_OPENAI=1
export OPENAI_BASE_URL=http://localhost:11434/v1
export OPENAI_MODEL=qwen2.5-coder:7b
```



openclaude

Windows PowerShell:

```
$env:CLAUDE_CODE_USE_OPENAI="1"
$env:OPENAI_BASE_URL="http://localhost:11434/v1"
$env:OPENAI_MODEL="qwen2.5-coder:7b"
```



openclaude

Setup Guides

Beginner-friendly guides:

- [Non-Technical Setup](#)
- [Windows Quick Start](#)
- [macOS / Linux Quick Start](#)

Advanced and source-build guides:

- [Advanced Setup](#)
- [Android Install](#)

Supported Providers

[README](#) [Code of conduct](#) [Contributing](#) [License](#) [Security](#)

Provider	Setup Path	Notes
OpenAI-compatible	<code>/provider</code> or env vars	Works with OpenAI, OpenRouter, DeepSeek, Groq, Mistral, LM Studio, and other compatible <code>/v1</code> servers
Hicap	<code>/provider</code> or OpenAI-compatible env vars	Uses <code>api-key</code> auth, discovers models from unauthenticated <code>/models</code> , and supports Responses mode for <code>gpt-</code> models
Gemini	<code>/provider</code> or env vars	Supports API key only
GitHub Models	<code>/onboard-github</code>	Interactive onboarding with saved credentials

Provider	Setup Path	Notes
Codex OAuth	<code>/provider</code>	Opens ChatGPT sign-in in your browser and stores Codex credentials securely
Codex	<code>/provider</code>	Uses existing Codex CLI auth, OpenClaude secure storage, or env credentials
Gitlawb Opengateway	<code>/provider</code> or zero-config fallback	Free smart gateway at <code>https://opengateway.gitlawb.com/v1</code> ; routes Xiaomi MiMo and GMI Cloud partner models by <code>OPENAI_MODEL</code>
Xiaomi MiMo	<code>/provider</code> or env vars	OpenAI-compatible API at <code>https://api.xiaomimimo.com/v1</code> ; uses <code>MIMO_API_KEY</code> and defaults to <code>mimo-v2.5-pro</code>
Ollama	<code>/provider</code> or env vars	Local inference with no API key
Atomic Chat	<code>/provider</code> , env vars, or <code>bun run dev:atomic-chat</code>	Local Model Provider; auto-detects loaded models
Bedrock / Vertex / Foundry	env vars	Additional provider integrations for supported environments

What Works

- **Tool-driven coding workflows:** Bash, file read/write/edit, grep, glob, agents, tasks, MCP, and slash commands
- **Streaming responses:** Real-time token output and tool progress
- **Tool calling:** Multi-step tool loops with model calls, tool execution, and follow-up responses
- **Images:** URL and base64 image inputs for providers that support vision
- **Provider profiles:** Guided setup plus saved user-level provider profile support
- **Local and remote model backends:** Cloud APIs, local servers, and Apple Silicon local inference

Provider Notes

OpenClaude supports multiple providers, but behavior is not identical across all of them.

- Anthropic-specific features may not exist on other providers
- Tool quality depends heavily on the selected model
- Smaller local models can struggle with long multi-step tool flows
- Some providers impose lower output caps than the CLI defaults, and OpenClaude adapts where possible
- Gitlawb Opengateway uses one OpenAI-compatible base URL. Switch between `mimo-*` and `google/gemini-3.1-flash-lite-preview` with `/model` ; do not pin the base URL to `/v1/xiaomi-mimo` .
- Xiaomi MiMo uses `api-key` header auth on the direct OpenAI-compatible route and currently does not support `/usage` reporting in OpenClaude

For best results, use models with strong tool/function calling support.

Agent Routing

OpenClaude can route different agents to different models through settings-based routing. This is useful for cost optimization or splitting wo by model strength.

Add to `~/.openclaude.json` :

```
{
  "agentModels": {
    "deepseek-v4-flash": {
      "base_url": "https://api.deepseek.com/v1",
      "api_key": "sk-your-key"
    },
    "gpt-4o": {
      "base_url": "https://api.openai.com/v1",
      "api_key": "sk-your-key"
    }
  },
  "agentRouting": {
    "Explore": "deepseek-v4-flash",
    "Plan": "gpt-4o",
    "general-purpose": "gpt-4o",
    "frontend-dev": "deepseek-v4-flash",
    "default": "gpt-4o"
  }
}
```



```
}  
}
```

When no routing match is found, the global provider remains the fallback.

Note: `api_key` values in `settings.json` are stored in plaintext. Keep this file private and do not commit it to version control.

Web Search and Fetch

By default, `webSearch` works on non-Anthropic models using DuckDuckGo. This gives GPT-4o, DeepSeek, Gemini, Ollama, and other OpenAI-compatible providers a free web search path out of the box.

Note: DuckDuckGo fallback works by scraping search results and may be rate-limited, blocked, or subject to DuckDuckGo's Terms of Service. If you want a more reliable supported option, configure Firecrawl.

For Anthropic-native backends and Codex responses, OpenClaude keeps the native provider web search behavior.

`webFetch` works, but its basic HTTP plus HTML-to-markdown path can still fail on JavaScript-rendered sites or sites that block plain HTTP requests.

Set a [Firecrawl](#) API key if you want Firecrawl-powered search/fetch behavior:

```
export FIRECRAWL_API_KEY=your-key-here
```



With Firecrawl enabled:

- `webSearch` can use Firecrawl's search API while DuckDuckGo remains the default free path for non-Claude models
- `webFetch` uses Firecrawl's scrape endpoint instead of raw HTTP, handling JS-rendered pages correctly

Free tier at [firecrawl.dev](#) includes 500 credits. The key is optional.

Headless gRPC Server

OpenClaude can be run as a headless gRPC service, allowing you to integrate its agentic capabilities (tools, bash, file editing) into other applications, CI/CD pipelines, or custom user interfaces. The server uses bidirectional streaming to send real-time text chunks, tool calls, and request permissions for sensitive commands.

1. Start the gRPC Server

Start the core engine as a gRPC service on `localhost:50051` :

```
npm run dev:grpc
```



Configuration

Variable	Default	Description
GRPC_PORT	50051	Port the gRPC server listens on
GRPC_HOST	localhost	Bind address. Use <code>0.0.0.0</code> to expose on all interfaces (not recommended without authentication)

2. Run the Test CLI Client

We provide a lightweight CLI client that communicates exclusively over gRPC. It acts just like the main interactive CLI, rendering colors, streaming tokens, and prompting you for tool permissions (y/n) via the gRPC `action_required` event.

In a separate terminal, run:

```
npm run dev:grpc:cli
```



Note: The gRPC definitions are located in `src/proto/openClaude.proto` . You can use this file to generate clients in Python, Go, Rust, or any other language.

Source Build And Local Development

```
bun install
bun run build
node dist/cli.mjs
```



Helpful commands:

- `bun run dev`
- `bun test`
- `bun run test:coverage`
- `bun run security:pr-scan -- --base origin/main`
- `bun run smoke`
- `bun run doctor:runtime`
- `bun run verify:privacy`
- `focused bun test ...` runs for the areas you touch

Testing And Coverage

OpenClaude uses Bun's built-in test runner for unit tests.

Run the full unit suite:

```
bun test
```



Generate unit test coverage:

```
bun run test:coverage
```



Open the visual coverage report:

```
open coverage/index.html
```



If you already have `coverage/lcov.info` and only want to rebuild the UI:

```
bun run test:coverage:ui
```



Use focused test runs when you only touch one area:

- `bun run test:provider`
- `bun run test:provider-recommendation`
- `bun test path/to/file.test.ts`

Recommended contributor validation before opening a PR:

- `bun run build`
- `bun run smoke`
- `bun run test:coverage` for broader unit coverage when your change affects shared runtime or provider logic
- `focused bun test ...` runs for the files and flows you changed

Coverage output is written to `coverage/lcov.info`, and OpenClaude also generates a git-activity-style heatmap at `coverage/index.html`.

Repository Structure

- `src/` - core CLI/runtime
- `scripts/` - build, verification, and maintenance scripts
- `docs/` - setup, contributor, and project documentation
- `python/` - standalone Python helpers and their tests
- `vscode-extension/openclaude-vscode/` - VS Code extension
- `.github/` - repo automation, templates, and CI configuration
- `bin/` - CLI launcher entrypoints

VS Code Extension

The repo includes a VS Code extension in [vscode-extension/openclaude-vscode](#) for OpenClaude launch integration, provider-aware control-center UI, and theme support.

Security

If you believe you found a security issue, see [SECURITY.md](#).

Community

- Use [GitHub Discussions](#) for Q&A, ideas, and community conversation
- Use [GitHub Issues](#) for confirmed bugs and actionable feature work

Contributing

Contributions are welcome.

For larger changes, open an issue first so the scope is clear before implementation. Helpful validation commands include:

- `bun run build`
- `bun run test:coverage`
- `bun run smoke`
- `focused bun test ...` runs for files and flows you changed

Disclaimer

OpenClaude is an independent community project and is not affiliated with, endorsed by, or sponsored by Anthropic.

OpenClaude originated from the Claude Code codebase and has since been substantially modified to support multiple providers and open us "Claude" and "Claude Code" are trademarks of Anthropic PBC. See [LICENSE](#) for details.

License

Releases 22

 **v0.14.0** Latest
3 days ago

[+ 21 releases](#)

Packages

No packages published

Contributors 116



[+ 102 contributors](#)

Languages

